

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**  
**KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO ĐỒ ÁN**

**HỆ THỐNG PHÂN TÁN**

**Đề tài: XÂY DỰNG ỨNG DỤNG MICROSERVICE BÁN HÀNG**

Giảng viên hướng dẫn :

TS. Kim Ngọc Bách

Lớp:

M25CQHT02-B

Sinh viên thực hiện :

B25CHHT084 - Nguyễn Công Đạt

B25CHHT094 - Trương Tuấn Hiệp

**Hà Nội - 2026**

# MỤC LỤC

|                                                     |    |
|-----------------------------------------------------|----|
| LỜI MỞ ĐẦU .....                                    | 4  |
| I. Mục tiêu đề án .....                             | 5  |
| 2. Mô tả bài toán .....                             | 5  |
| 3. Kiến trúc hệ thống.....                          | 6  |
| 3.1. Tổng quan kiến trúc.....                       | 6  |
| 3.2. Các thành phần chính .....                     | 6  |
| 3.3. Cổng và database .....                         | 7  |
| 4. Mô tả các microservice .....                     | 8  |
| 4.1. API Gateway.....                               | 8  |
| 4.2. User Service.....                              | 8  |
| 4.3. Product Service.....                           | 9  |
| 4.4. Order Service.....                             | 9  |
| 4.5. Inventory Service.....                         | 10 |
| 4.6. Notification Service.....                      | 10 |
| 4.7. Common Lib.....                                | 10 |
| 5. Thiết kế RESTful API.....                        | 11 |
| 5.1. Quy ước response .....                         | 11 |
| 5.2. API xác thực và người dùng .....               | 11 |
| 5.3. API sản phẩm và danh mục .....                 | 12 |
| 5.4. API đơn hàng .....                             | 13 |
| 5.5. API tồn kho .....                              | 14 |
| 5.6. API thông báo .....                            | 15 |
| 5.7. API thanh toán .....                           | 15 |
| 6. Luồng Message Queue.....                         | 15 |
| 6.1. Topic và consumer group .....                  | 15 |
| 6.2. Event type.....                                | 16 |
| 6.3. Luồng tạo đơn hàng.....                        | 16 |
| 6.4. Luồng hủy đơn hàng.....                        | 17 |
| 7. Tính mở rộng, chịu lỗi và nhất quán dữ liệu..... | 18 |
| 7.1. Tính mở rộng .....                             | 18 |

|                                         |    |
|-----------------------------------------|----|
| 7.2. Chịu lỗi .....                     | 18 |
| 7.3. Nhất quán dữ liệu .....            | 18 |
| 8. Thử nghiệm và đánh giá .....         | 19 |
| 8.1. Môi trường thử nghiệm .....        | 19 |
| 8.2. Kịch bản thử nghiệm chức năng..... | 19 |
| 8.3. Demo một số kết quả.....           | 21 |
| 8.3.1. Đăng nhập / Đăng kí .....        | 21 |
| 8.3.2. Đặt hàng .....                   | 21 |
| 8.3.3. Thanh toán.....                  | 22 |
| 8.4. Đánh giá kết quả.....              | 22 |
| 8.5. Hạn chế và hướng phát triển.....   | 23 |
| 9. Kết luận.....                        | 24 |

## LỜI MỞ ĐẦU

Trong bối cảnh các hệ thống phần mềm ngày càng cần khả năng mở rộng, triển khai linh hoạt và chịu lỗi tốt, kiến trúc microservice trở thành một hướng tiếp cận quan trọng. Báo cáo này trình bày quá trình xây dựng một ứng dụng bán hàng theo kiến trúc microservice, sử dụng RESTful API cho giao tiếp đồng bộ và Apache Kafka cho giao tiếp bất đồng bộ.

Nội dung báo cáo tập trung vào mô tả bài toán, kiến trúc hệ thống, vai trò của từng microservice, thiết kế RESTful API, luồng message queue, cũng như đánh giá khả năng mở rộng, chịu lỗi và nhất quán dữ liệu của hệ thống.

## I. Mục tiêu đề án

Đề án xây dựng một ứng dụng theo kiến trúc microservice, trong đó các chức năng nghiệp vụ được tách thành nhiều dịch vụ độc lập. Mỗi dịch vụ có phạm vi trách nhiệm riêng, có thể phát triển, triển khai và mở rộng độc lập.

Hệ thống sử dụng hai hình thức giao tiếp liên dịch vụ:

- RESTful API cho các thao tác đồng bộ, khi dịch vụ gọi cần kết quả ngay để tiếp tục xử lý.
- Message Queue thông qua Apache Kafka cho các thao tác bất đồng bộ, giúp giảm phụ thuộc trực tiếp giữa các dịch vụ và tăng khả năng chịu lỗi.

Mục tiêu chính của đề án là vận dụng các kiến thức về phân tách dịch vụ, giao tiếp liên dịch vụ, tính mở rộng, khả năng chịu lỗi và nhất quán dữ liệu trong hệ phân tán.

## 2. Mô tả bài toán

Hệ thống mô phỏng một ứng dụng bán hàng trực tuyến với các chức năng cơ bản:

- Người dùng có thể đăng ký, đăng nhập.
- Khách hàng có thể xem danh sách sản phẩm và danh mục sản phẩm.
- Người dùng đã đăng nhập có thể tạo đơn hàng từ các sản phẩm có trong hệ thống.
- Hệ thống quản lý tồn kho theo từng sản phẩm.
- Khi đơn hàng được tạo hoặc hủy, tồn kho được cập nhật thông qua sự kiện bất đồng bộ.
- Hệ thống tạo thông báo cho người dùng khi đơn hàng được tạo/hủy và tạo cảnh báo cho quản trị viên khi hàng tồn kho thấp.
- Người dùng có thể thanh toán cho các sản phẩm đã đặt hàng hoặc hủy thanh toán, khi thanh toán thì tiền ở tài khoản người dùng sẽ bị trừ số dư tương ứng.

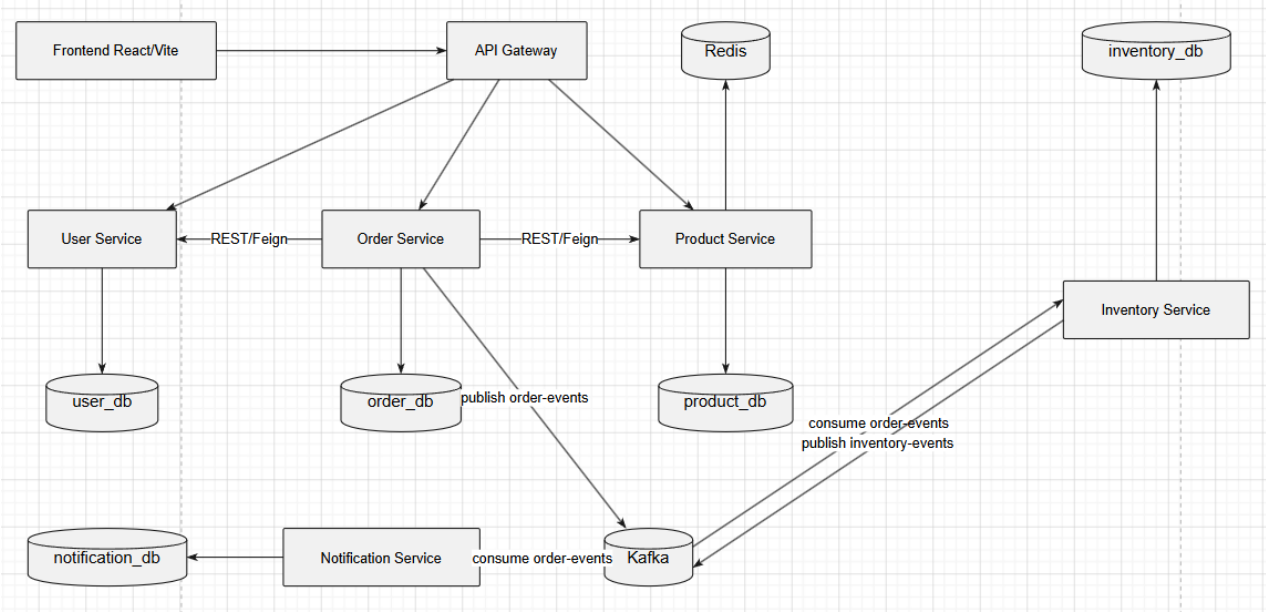
Bài toán phù hợp với kiến trúc microservice vì các miền nghiệp vụ như người dùng, sản phẩm, đơn hàng, tồn kho, số dư và thông báo có thể tách rời tương đối rõ ràng. Việc tách dịch vụ giúp giảm sự phụ thuộc giữa các module, đồng thời cho phép từng dịch vụ có database riêng và vòng đời triển khai riêng.

### 3. Kiến trúc hệ thống

#### 3.1. Tổng quan kiến trúc

Hệ thống gồm frontend React/Vite, API Gateway, các microservice nghiệp vụ, PostgreSQL, Redis và Kafka.

Sơ đồ tổng quan kiến trúc:



#### 3.2. Các thành phần chính

| Thành phần   | Công nghệ                      | Vai trò                                                                                                                     |
|--------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Frontend     | React/Vite                     | Giao diện người dùng, gọi API qua Gateway                                                                                   |
| API Gateway  | Spring Cloud Gateway           | Định tuyến request, kiểm tra JWT, thêm header người dùng cho service phía sau                                               |
| User Service | Spring Boot, PostgreSQL, Redis | Quản lý đăng ký, đăng nhập, JWT và hồ sơ người dùng, quản lý số dư tài khoản, cung cấp API nội bộ để thực hiện trừ tiền khi |

|                      |                                                    |                                                                      |
|----------------------|----------------------------------------------------|----------------------------------------------------------------------|
|                      |                                                    | mua hàng và hoàn tiền khi hủy mua hàng                               |
| Product Service      | Spring Boot, PostgreSQL, Redis                     | Quản lý sản phẩm và danh mục                                         |
| Order Service        | Spring Boot, PostgreSQL, OpenFeign, Kafka Producer | Quản lý đơn hàng, gọi Product Service đồng bộ, phát sự kiện đơn hàng |
| Inventory Service    | Spring Boot, PostgreSQL, Kafka Consumer/Producer   | Quản lý tồn kho, xử lý sự kiện đơn hàng, phát sự kiện tồn kho        |
| Notification Service | Spring Boot, PostgreSQL, Kafka Consumer            | Lưu và cung cấp thông báo cho người dùng                             |
| Kafka                | Apache Kafka                                       | Message Queue cho giao tiếp bất đồng bộ                              |
| PostgreSQL           | PostgreSQL 16                                      | Lưu trữ dữ liệu riêng cho từng service                               |
| Redis                | Redis 7                                            | Hỗ trợ cache/lưu dữ liệu phụ trợ cho một số service                  |

### 3.3. Cổng và database

| Service              | Port nội bộ | Database                |
|----------------------|-------------|-------------------------|
| API Gateway          | 8080        | Không có database riêng |
| User Service         | 8081        | user_db                 |
| Product Service      | 8082        | product_db              |
| Order Service        | 8083        | order_db                |
| Inventory Service    | 8084        | inventory_db            |
| Notification Service | 8085        | notification_db         |

## 4. Mô tả các microservice

### 4.1. API Gateway

API Gateway là điểm vào duy nhất cho frontend. Gateway chịu trách nhiệm định tuyến request tới service phù hợp dựa trên URL.

Các API chính:

- /api/auth/\*\* được chuyển tới User Service và không yêu cầu JWT.
- /api/users/\*\* được chuyển tới User Service và yêu cầu JWT.
- GET /api/products/\*\*, GET /api/categories/\*\* là public.
- Các API tạo/sửa/xóa sản phẩm, đơn hàng, tồn kho và thông báo yêu cầu JWT.

Khi request có JWT hợp lệ, Gateway trích xuất thông tin người dùng và thêm vào header:

- X-User-Id
- X-User-Role

Các service phía sau sử dụng header này để biết người dùng đang thao tác.

### 4.2. User Service

User Service quản lý người dùng và xác thực. Service này cung cấp chức năng đăng ký, đăng nhập và truy vấn/cập nhật thông tin người dùng. Ngoài ra còn lưu thông tin số dư tài khoản, cung cấp API nội bộ thực hiện xử lý thanh toán như trừ tiền khi mua hàng và hoàn tiền khi hủy đơn hàng.

Trách nhiệm chính:

- Tạo tài khoản người dùng mới.
- Xác thực username/password.
- Sinh JWT sau khi đăng nhập thành công.
- Trả về thông tin người dùng hiện tại.
- Cập nhật hồ sơ người dùng.
- Thanh toán đơn hàng



Dữ liệu người dùng được lưu trong database user\_db.

### 4.3. Product Service

Product Service quản lý sản phẩm và danh mục sản phẩm.

Trách nhiệm chính:

- Lấy danh sách sản phẩm có phân trang.
- Lấy chi tiết sản phẩm theo ID.
- Tạo, cập nhật và xóa sản phẩm.
- Quản lý danh mục sản phẩm.

Order Service gọi Product Service bằng REST API đồng bộ để lấy thông tin sản phẩm khi tạo đơn hàng. Điều này đảm bảo đơn hàng sử dụng đúng tên, giá và trạng thái active của sản phẩm tại thời điểm tạo đơn.

### 4.4. Order Service

Order Service quản lý vòng đời đơn hàng.

Trách nhiệm chính:

- Tạo đơn hàng cho người dùng.
- Lấy danh sách đơn hàng của người dùng.
- Lấy chi tiết đơn hàng.
- Cập nhật trạng thái đơn hàng.
- Hủy đơn hàng.
- Phát sự kiện Kafka khi đơn hàng được tạo hoặc bị hủy.

Khi tạo đơn hàng, Order Service gọi Product Service thông qua OpenFeign để kiểm tra sản phẩm. Sau khi đơn hàng được lưu vào order\_db, service phát sự kiện ORDER\_CREATED lên topic order-events.

Khi hủy đơn hàng, service cập nhật trạng thái đơn hàng thành CANCELLED và phát sự kiện ORDER\_CANCELLED.

## 4.5. Inventory Service

Inventory Service quản lý số lượng tồn kho theo sản phẩm.

Trách nhiệm chính:

- Xem toàn bộ tồn kho.
- Xem tồn kho theo productId.
- Cập nhật số lượng tồn kho thủ công.
- Lắng nghe sự kiện đơn hàng để trừ hoặc hoàn lại tồn kho.
- Phát sự kiện tồn kho khi số lượng được cập nhật hoặc khi tồn kho thấp.

Inventory Service lắng nghe topic order-events:

- Với ORDER\_CREATED, service trừ tồn kho theo các item trong đơn hàng.
- Với ORDER\_CANCELLED, service hoàn lại tồn kho.

Khi tồn kho còn nhỏ hơn hoặc bằng 5, service phát sự kiện INVENTORY\_UPDATED lên topic inventory-events để Notification Service tạo cảnh báo.

## 4.6. Notification Service

Notification Service xử lý thông báo cho người dùng và quản trị viên.

Trách nhiệm chính:

- Lắng nghe sự kiện đơn hàng từ topic order-events.
- Lắng nghe sự kiện tồn kho từ topic inventory-events.
- Tạo thông báo khi đơn hàng được tạo hoặc hủy.
- Tạo cảnh báo tồn kho thấp cho quản trị viên.
- Cung cấp API để người dùng xem thông báo và đánh dấu đã đọc.

Thông báo được lưu trong database notification\_db.

## 4.7. Common Lib

common-lib chứa các thành phần dùng chung giữa các service:

- ApiResponse: định dạng response thống nhất.
- OrderEvent, InventoryEvent, BaseEvent: mô hình event dùng qua Kafka.

- KafkaConstants: tên topic, consumer group và event type.
- Exception dùng chung như BadRequestException, ResourceNotFoundException.

## 5. Thiết kế RESTful API

### 5.1. Quy ước response

Các API trả về dữ liệu theo cấu trúc chung:

*Ví dụ:*

```
{
  "success": true,
  "data": {},
  "message": "Optional message"
}
```

Khi có lỗi, hệ thống trả về response có success = false và message mô tả lỗi.

### 5.2. API xác thực và người dùng

| Method | Endpoint           | Mô tả                                  | Yêu cầu JWT |
|--------|--------------------|----------------------------------------|-------------|
| POST   | /api/auth/register | Đăng ký tài khoản                      | Không       |
| POST   | /api/auth/login    | Đăng nhập và nhận JWT                  | Không       |
| GET    | /api/users/me      | Lấy thông tin người dùng hiện tại      | Có          |
| GET    | /api/users/{id}    | Lấy thông tin người dùng theo ID       | Có          |
| PUT    | /api/users/me      | Cập nhật thông tin người dùng hiện tại | Có          |

Ví dụ đăng ký:

*Ví dụ:*

```
{
  "username": "student01",
  "email": "student01@example.com",
  "password": "123456"
}
```

Ví dụ đăng nhập:

*Ví dụ:*

```
{
  "username": "student01",
  "password": "123456"
}
```

### 5.3. API sản phẩm và danh mục

| Method | Endpoint           | Mô tả                                | Yêu cầu JWT |
|--------|--------------------|--------------------------------------|-------------|
| GET    | /api/products      | Lấy danh sách sản phẩm có phân trang | Không       |
| GET    | /api/products/{id} | Lấy chi tiết sản phẩm                | Không       |
| POST   | /api/products      | Tạo sản phẩm                         | Có          |
| PUT    | /api/products/{id} | Cập nhật sản phẩm                    | Có          |
| DELETE | /api/products/{id} | Xóa sản phẩm                         | Có          |
| GET    | /api/categories    | Lấy danh sách danh mục               | Không       |
| POST   | /api/categories    | Tạo danh mục                         | Có          |

Ví dụ tạo sản phẩm:

*Ví dụ:*

```
{
  "name": "Laptop Dell",

```

```
"description": "Laptop phục vụ học tập và làm việc",
"price": 15000000,
"categoryId": 1,
"imageUrl": "https://example.com/laptop.jpg"
}
```

Ví dụ tạo danh mục:

*Ví dụ:*

```
{
  "name": "Laptop",
  "description": "Thiết bị máy tính xách tay"
}
```

#### 5.4. API đơn hàng

| Method | Endpoint                | Mô tả                                          | Yêu cầu JWT |
|--------|-------------------------|------------------------------------------------|-------------|
| POST   | /api/orders             | Tạo đơn hàng                                   | Có          |
| GET    | /api/orders             | Lấy danh sách đơn hàng của người dùng hiện tại | Có          |
| GET    | /api/orders/{id}        | Lấy chi tiết đơn hàng                          | Có          |
| PUT    | /api/orders/{id}/status | Cập nhật trạng thái đơn hàng                   | Có          |
| DELETE | /api/orders/{id}        | Hủy đơn hàng                                   | Có          |

Ví dụ tạo đơn hàng:

*Ví dụ:*

```
{
  "items": [
    {
```

```

    "productId": 1,
    "quantity": 2
  },
  {
    "productId": 3,
    "quantity": 1
  }
]
}

```

Ví dụ cập nhật trạng thái:

*Ví dụ:*

```

{
  "status": "PAID"
}

```

## 5.5. API tồn kho

| Method | Endpoint                   | Mô tả                     | Yêu cầu JWT |
|--------|----------------------------|---------------------------|-------------|
| GET    | /api/inventory             | Lấy toàn bộ tồn kho       | Có          |
| GET    | /api/inventory/{productId} | Lấy tồn kho theo sản phẩm | Có          |
| PUT    | /api/inventory/{productId} | Cập nhật số lượng tồn kho | Có          |

Ví dụ cập nhật tồn kho:

*Ví dụ:*

```

{
  "quantity": 100
}

```

## 5.6. API thông báo

| Method | Endpoint                     | Mô tả                                           | Yêu cầu JWT |
|--------|------------------------------|-------------------------------------------------|-------------|
| GET    | /api/notifications           | Lấy danh sách thông báo của người dùng hiện tại | Có          |
| PUT    | /api/notifications/{id}/read | Đánh dấu thông báo đã đọc                       | Có          |

## 5.7. API thanh toán

| Method | Endpoint                       | Mô tả                                         | Yêu cầu JWT |
|--------|--------------------------------|-----------------------------------------------|-------------|
| PUT    | /api/users/{id}/deduct-balance | Trừ tiền vào tài khoản khi thanh toán         | Không       |
| PUT    | /api/users/{id}/refund-balance | Hoàn tiền nếu hủy đơn hàng hoặc giao dịch lỗi | Không       |

## 6. Luồng Message Queue

### 6.1. Topic và consumer group

| Topic        | Producer      | Consumer                                | Mục đích                                    |
|--------------|---------------|-----------------------------------------|---------------------------------------------|
| order-events | Order Service | Inventory Service, Notification Service | Phát sự kiện khi đơn hàng được tạo hoặc hủy |

|                  |                   |                      |                                                          |
|------------------|-------------------|----------------------|----------------------------------------------------------|
| inventory-events | Inventory Service | Notification Service | Phát sự kiện khi tồn kho được cập nhật hoặc tồn kho thấp |
|------------------|-------------------|----------------------|----------------------------------------------------------|

Consumer group:

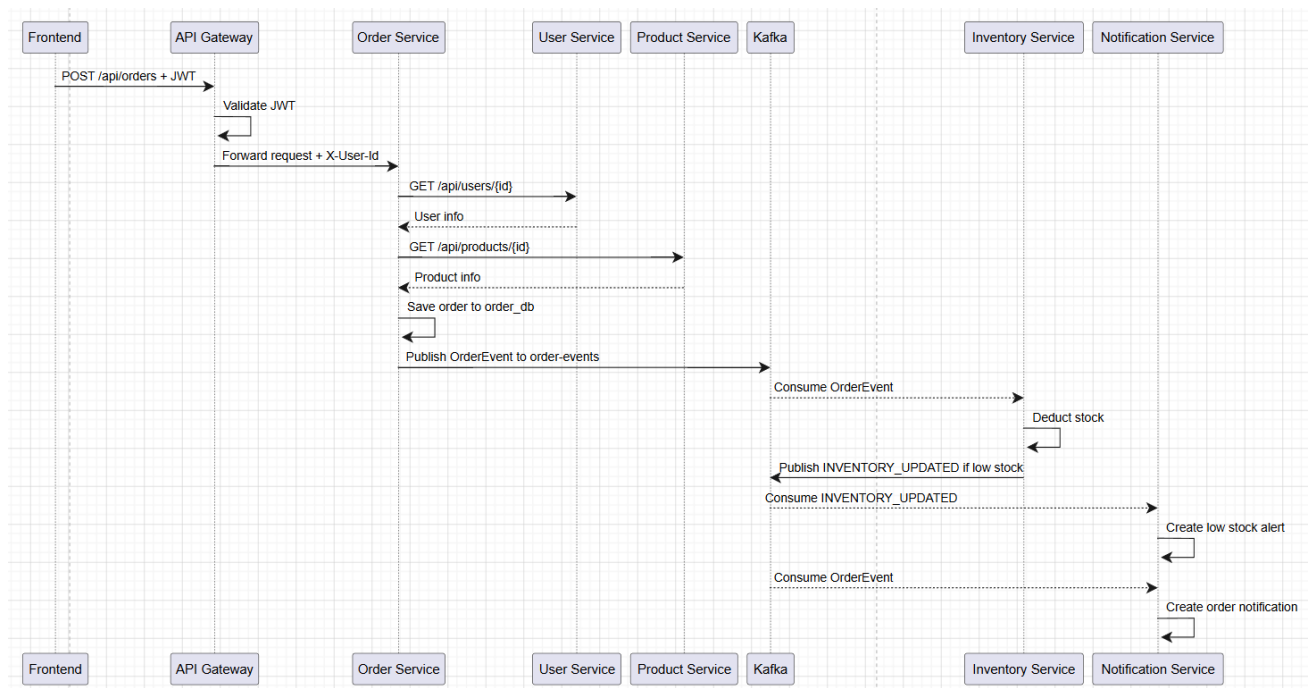
- inventory-group: Inventory Service xử lý sự kiện đơn hàng.
- notification-order-group: Notification Service xử lý sự kiện đơn hàng.
- notification-inventory-group: Notification Service xử lý sự kiện tồn kho.

## 6.2. Event type

| Event type        | Nguồn phát        | Ý nghĩa               |
|-------------------|-------------------|-----------------------|
| ORDER_CREATED     | Order Service     | Đơn hàng mới được tạo |
| ORDER_CANCELLED   | Order Service     | Đơn hàng bị hủy       |
| INVENTORY_UPDATED | Inventory Service | Tồn kho được cập nhật |

## 6.3. Luồng tạo đơn hàng

Sequence diagram:



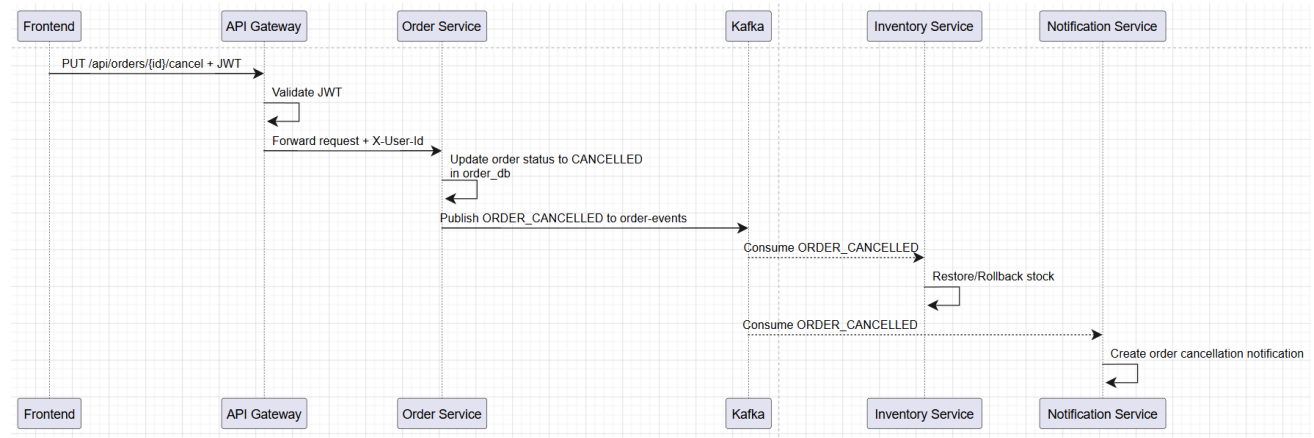


Mô tả:

1. Người dùng gửi request tạo đơn hàng qua API Gateway.
2. Gateway kiểm tra JWT và truyền X-User-Id xuống Order Service.
3. Order Service gọi Product Service bằng REST API để lấy thông tin sản phẩm.
4. Order Service lưu đơn hàng vào database riêng.
5. Order Service phát sự kiện ORDER\_CREATED lên Kafka.
6. Inventory Service nhận sự kiện và trừ tồn kho.
7. Notification Service nhận sự kiện và tạo thông báo đơn hàng.
8. Nếu tồn kho thấp, Inventory Service phát thêm sự kiện INVENTORY\_UPDATED.
9. Notification Service tạo cảnh báo tồn kho thấp cho quản trị viên.

## 6.4. Luồng hủy đơn hàng

Sequence diagram:



Mô tả:

1. Người dùng yêu cầu hủy đơn hàng.
2. Order Service cập nhật trạng thái đơn hàng thành CANCELLED.
3. Order Service phát sự kiện ORDER\_CANCELLED.
4. Inventory Service nhận sự kiện và hoàn lại tồn kho.
5. Notification Service tạo thông báo đơn hàng đã hủy cho người dùng.

## **7. Tính mở rộng, chịu lỗi và nhất quán dữ liệu**

### **7.1. Tính mở rộng**

Kiến trúc microservice cho phép mở rộng từng service độc lập. Ví dụ:

- Nếu lượng truy cập xem sản phẩm tăng cao, có thể scale riêng Product Service.
- Nếu số lượng đơn hàng tăng, có thể scale Order Service và Inventory Service.
- Kafka giúp xử lý bất đồng bộ, giảm áp lực xử lý tức thời lên các service phụ thuộc.

### **7.2. Chịu lỗi**

Hệ thống giảm phụ thuộc trực tiếp bằng cách dùng Kafka cho các nghiệp vụ hậu xử lý. Khi Order Service tạo đơn hàng thành công, việc cập nhật tồn kho và gửi thông báo được xử lý bất đồng bộ. Nếu Notification Service tạm thời gặp lỗi, sự kiện vẫn có thể được xử lý lại từ Kafka tùy cấu hình consumer.

Inventory Service sử dụng cơ chế retry cho consumer xử lý sự kiện đơn hàng. Điều này giúp tăng khả năng xử lý lại khi gặp lỗi tạm thời.

### **7.3. Nhất quán dữ liệu**

Mỗi service sở hữu database riêng, vì vậy hệ thống không dùng transaction phân tán giữa nhiều database. Thay vào đó, hệ thống hướng đến mô hình eventual consistency:

- Order Service lưu đơn hàng trước.
- Sau đó sự kiện Kafka được phát ra.
- Inventory Service và Notification Service cập nhật dữ liệu của mình dựa trên sự kiện.

Cách tiếp cận này phù hợp với hệ phân tán vì tránh khóa transaction trên nhiều service, nhưng cần chấp nhận độ trễ ngắn giữa thời điểm đơn hàng được tạo và thời điểm tồn kho/thông báo được cập nhật.

## 8. Thử nghiệm và đánh giá

### 8.1. Môi trường thử nghiệm

Hệ thống được triển khai trên cụm Kubernetes (K8s) thông qua các manifest YAML.

Môi trường được phân bổ bao gồm các thành phần sau:

- Namespace và Secrets: Khởi tạo không gian mạng và quản lý biến môi trường bảo mật thông qua k8s/namespace-and-secrets.yaml
- Infrastructure (Cơ sở hạ tầng): Triển khai thông qua thư mục k8s/infra/, bao gồm PostgreSQL (postgres.yaml), Redis (redis.yaml), cùng cụm Kafka và Zookeeper (kafka-zookeeper.yaml)
- API Gateway: Được định nghĩa và triển khai độc lập thông qua k8s/services/api-gateway.yaml
- Microservices: Các dịch vụ backend (User, Product, Order, Inventory, Notification) được đóng gói và cấu hình triển khai trong k8s/services/microservices.yaml
- Frontend: Ứng dụng React/Vite được deploy thông qua k8s/frontend/frontend.yaml

### 8.2. Kịch bản thử nghiệm chức năng

| STT | Kịch bản                          | Kết quả mong đợi                            |
|-----|-----------------------------------|---------------------------------------------|
| 1   | Đăng ký tài khoản mới             | Tài khoản được tạo, API trả về thành công   |
| 2   | Đăng nhập                         | API trả về JWT                              |
| 3   | Gọi API cần xác thực không có JWT | Gateway trả về 401 Unauthorized             |
| 4   | Lấy danh sách sản phẩm            | API trả về danh sách sản phẩm có phân trang |
| 5   | Tạo sản phẩm mới                  | Product Service lưu sản phẩm vào product_db |

|    |                           |                                                                        |
|----|---------------------------|------------------------------------------------------------------------|
| 6  | Cập nhật tồn kho sản phẩm | Inventory Service cập nhật số lượng trong inventory_db                 |
| 7  | Tạo đơn hàng              | Order Service lưu đơn hàng và phát ORDER_CREATED                       |
| 8  | Sau khi tạo đơn hàng      | Inventory Service trừ tồn kho tương ứng                                |
| 9  | Sau khi tạo đơn hàng      | Notification Service tạo thông báo cho người dùng                      |
| 10 | Hủy đơn hàng              | Order Service cập nhật trạng thái và phát ORDER_CANCELLED              |
| 11 | Sau khi hủy đơn hàng      | Inventory Service hoàn lại tồn kho                                     |
| 12 | Khi tồn kho thấp          | Notification Service tạo cảnh báo cho quản trị viên                    |
| 13 | Thanh toán                | Số dư sẽ giảm nếu thực hiện thanh toán và sẽ hoàn trả nếu hủy đơn hàng |

## 8.3. Demo một số kết quả

### 8.3.1. Đăng nhập / Đăng kí

## Login / Register

Username

Password

Login Register

### 8.3.2. Đặt hàng

E-CommerceProductsOrders

Vì: 10.000.000 đLogout

#### Products



ThinkPad X1 Carbon

450.000 đ

Buy Now



Dell XPS 15

550.000 đ

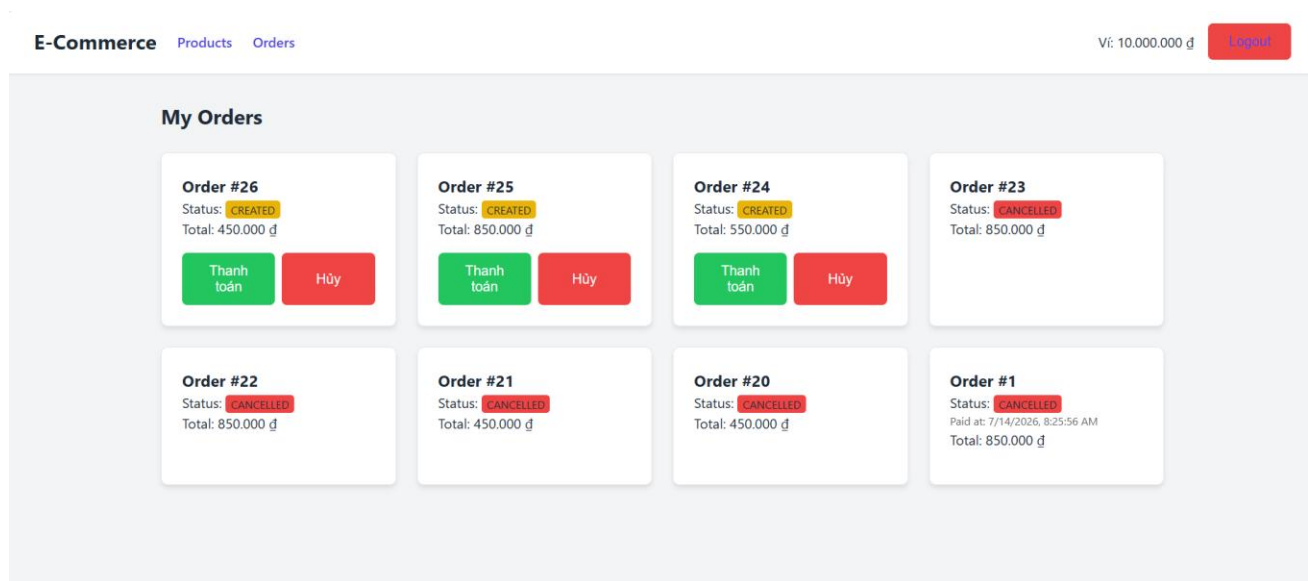
Buy Now



MacBook Pro 16

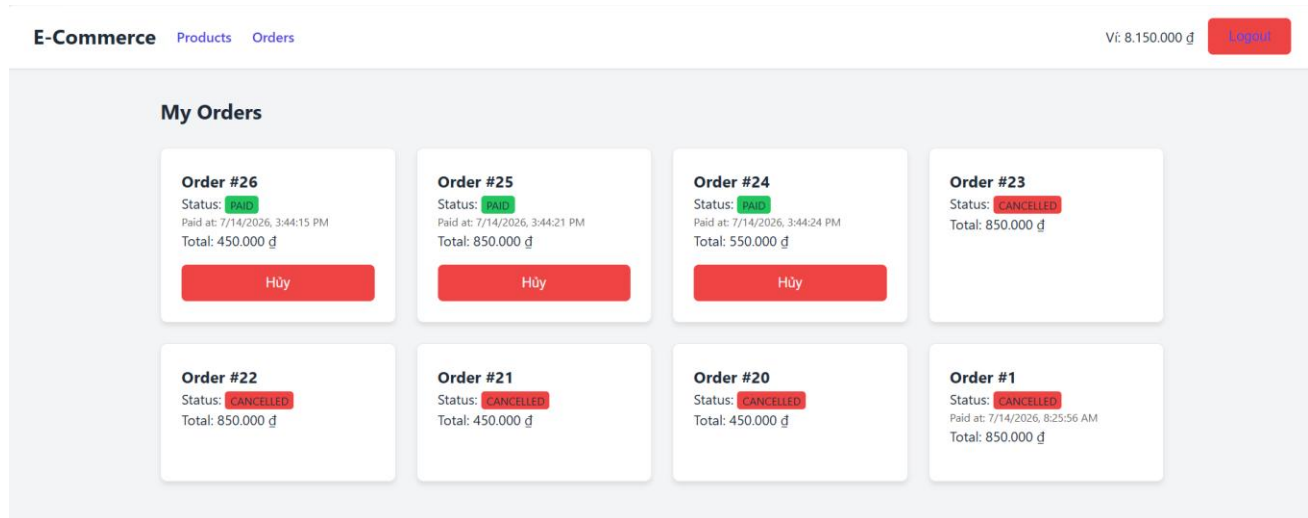
850.000 đ

Buy Now



Các đơn đã đặt hàng sẽ nằm ở mục orders, lúc này thì ở database tồn kho sẽ bị trừ đi số lượng tương ứng với sản phẩm đã đặt.

### 8.3.3. Thanh toán



Lúc này khi bấm thanh toán số dư tài khoản sẽ bị trừ tương ứng hoặc hoàn trả khi bấm hủy.

### 8.4. Đánh giá kết quả

Hệ thống đáp ứng các yêu cầu chính của đề án:

- Các service được tách theo miền nghiệp vụ rõ ràng.
- Mỗi service có database riêng, hạn chế chia sẻ dữ liệu trực tiếp.

- RESTful API được sử dụng cho giao tiếp đồng bộ, đặc biệt trong luồng tạo đơn hàng cần kiểm tra sản phẩm.
- Kafka được sử dụng cho giao tiếp bất đồng bộ giữa Order, Inventory và Notification Service.
- API Gateway đóng vai trò điểm vào tập trung, định tuyến request và xử lý xác thực JWT.
- Hệ thống có khả năng mở rộng từng service độc lập.
- Dữ liệu giữa các service được đồng bộ theo mô hình eventual consistency.

### 8.5. Hạn chế và hướng phát triển

Một số hạn chế hiện tại:

- Chưa có service discovery như Eureka hoặc Consul; địa chỉ service đang được cấu hình qua biến môi trường.
- Chưa có cơ chế distributed tracing để theo dõi request đi qua nhiều service.
- Chưa có Circuit Breaker cho các REST call đồng bộ giữa service.
- Chưa có cơ chế phân quyền chi tiết theo role cho từng API quản trị.
- Chưa có outbox pattern để đảm bảo chặt chẽ hơn giữa thao tác lưu database và phát Kafka event.

Hướng phát triển:

- Bổ sung Spring Cloud Circuit Breaker hoặc Resilience4j.
- Bổ sung centralized logging và distributed tracing.
- Áp dụng outbox pattern cho các event quan trọng.
- Bổ sung monitoring cho Kafka consumer lag và health check của từng service.
- Hoàn thiện phân quyền theo vai trò người dùng/quản trị viên.
- Viết thêm test tự động cho từng service và test tích hợp cho các luồng Kafka.

## 9. Kết luận

Đồ án đã xây dựng một hệ thống bán hàng theo kiến trúc microservice với các dịch vụ được phân tách độc lập theo nghiệp vụ. Hệ thống kết hợp RESTful API cho giao tiếp đồng bộ và Apache Kafka cho giao tiếp bất đồng bộ, qua đó thể hiện được các đặc trưng quan trọng của hệ phân tán như triển khai độc lập, mở rộng độc lập, chịu lỗi và nhất quán dữ liệu cuối cùng.

Thông qua đồ án, sinh viên có thể hiểu rõ hơn cách thiết kế ranh giới service, cách tổ chức database riêng cho từng service, cách sử dụng API Gateway để tập trung định tuyến/xác thực và cách dùng message queue để giảm phụ thuộc trực tiếp giữa các thành phần trong hệ thống.